



南开大学
Nankai University

南 开 大 学

数据安全实验报告

实验 3：秘密共享实践

学院：密码与网络空间安全学院

专业：物联网工程

学号：2310422

姓名：谢小珂

指导教师：刘哲理

2026 年 5 月 21 日

目录

1 实验要求	1
2 实验原理	1
2.1 Shamir (t, n) 门限秘密共享	1
2.2 多项式加法同态与隐私均值计算	2
2.3 复杂数据的有限域编码与解码	2
3 协议设计与形式化描述	3
3.1 实体角色定义	3
3.2 协议执行步骤	3
4 实验代码	5
4.1 核心类结构与职责	5
4.2 关键函数实现	5
4.2.1 模逆元计算 (mod_inverse)	5
4.2.2 拉格朗日插值 (verbose_lagrange)	6
4.2.3 数据所有者初始化与多项式构造	6
4.2.4 份额生成与本地求和	6
4.2.5 负数解码与去缩放	7
5 实验步骤与结果分析	7
5.1 协议执行步骤	7
5.1.1 步骤一：数据预处理与有限域自适应编码	7
5.1.2 步骤二：数据所有者本地盲化与安全多项式构造	8
5.1.3 步骤三：秘密份额跨网络分发	8
5.1.4 步骤四：计算节点内部数据隔离审计	9
5.1.5 步骤五：节点本地独立同态线性聚合	10
5.2 门限阈值恢复与数学重构推导 (C_3^2 组合验证分析)	10
5.3 符号还原、去缩放与最终均值计算	11
5.4 权威明文基准对照结论	11
6 实验遇到的问题	12
6.1 浮点数与有限域的不兼容	12
6.2 负数在有限域中的表示与还原	12
6.3 伪随机数生成器的安全性	12
6.4 有限域中除法的实现	12
7 实验总结	13

1 实验要求

借鉴实验 3.2，实现三个人对于他们拥有的数据的平均值的计算。

实验3.2 假设有三个同学需要对班里的优秀干部Alice、Bob、Charles、Douglas进行投票，最后统计各个班干部获得的票数。这个时候就可以利用Shamir秘密共享将各个投票方的投票分享出去并进行隐私求和计算。

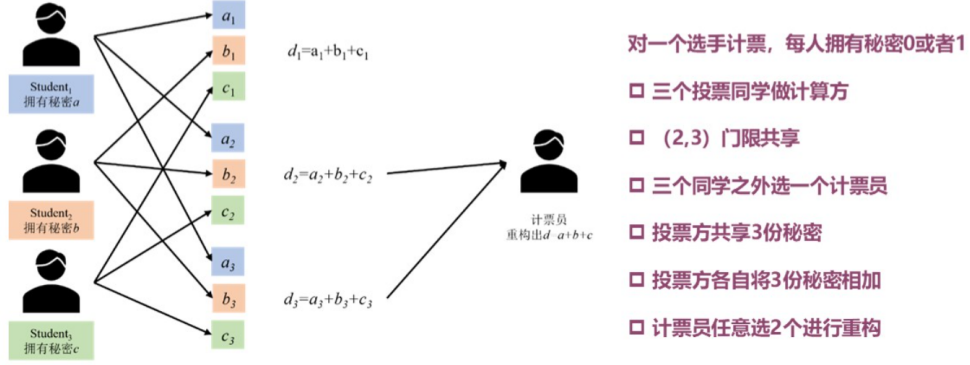


图 1: 投票统计示例

2 实验原理

2.1 Shamir (t, n) 门限秘密共享

根据密码学中 Shamir 门限方案的学术定义， (t, n) 门限方案是基于拉格朗日插值法构造的。其核心思想：在有限域上，通过 t 个不同的点可以唯一确定一个次数不超过 $t - 1$ 的多项式；而若持有的点数少于 t 个，则多项式的常数项（即秘密）在信息论意义下完全不可知。

秘密分配阶段 (Shr): 设秘密值为 s ，将其映射到大素数 P 构成的有限域 $\mathbb{F}_P$ 中。随机选择 $t - 1$ 个有限域内的系数 $a_1, a_2, \dots, a_{t-1} \in \mathbb{F}_P$ ，构造一个 $t - 1$ 次多项式：

$$f(x) = s + a_1x + a_2x^2 + \dots + a_{t-1}x^{t-1} \pmod{P}.$$

公开 n 个互不相同的非零横坐标 $x_1, x_2, \dots, x_n$ ，计算对应的纵坐标 $y_i = f(x_i)$ ，并将份额 (x_i, y_i) 分发给参与方 P_i 。显然，多项式的常数项即为秘密：

$$f(0) = s.$$

秘密重构阶段 (Rec): 任意 t 个参与方汇集其份额，在有限域 $\mathbb{F}_P$ 上利用拉格朗日插值公式恢复多项式 $f(x)$ 并求解 $f(0)$ 。拉格朗日插值公式在 $x = 0$ 处的形式为：

$$s = f(0) = \sum_{i=1}^t y_i \cdot \Delta_i(0) \pmod{P},$$

其中 $\Delta_i(0)$ 为拉格朗日基函数在 0 处的系数，在有限域中计算为：

$$\Delta_i(0) = \prod_{\substack{j=1 \\ j \neq i}}^t \frac{-x_j}{x_i - x_j} \pmod{P}.$$

2.2 多项式加法同态与隐私均值计算

本实验利用多项式在加法运算下的同态性质,使得计算节点可以直接在密文份额上进行线性组合,无需预先解密。

设有 $M = 3$ 个数据拥有者 (记作 A, B, C), 分别持有秘密 s_A, s_B, s_C 。在 $(2, 3)$ 门限下 (即 $t = 2, n = 3$), 每一方独立构造一个一次随机多项式:

$$f_A(x) = s_A + a_1x \pmod{P}, \quad f_B(x) = s_B + b_1x \pmod{P}, \quad f_C(x) = s_C + c_1x \pmod{P}.$$

系统设 3 个计算节点, 其公开横坐标分别为 x_1, x_2, x_3 。数据拥有者将份额横向分发给对应节点:

- 节点 C_1 (横坐标 x_1) 收到: $f_A(x_1), f_B(x_1), f_C(x_1)$;
- 节点 C_2 (横坐标 x_2) 收到: $f_A(x_2), f_B(x_2), f_C(x_2)$;
- 节点 C_3 (横坐标 x_3) 收到: $f_A(x_3), f_B(x_3), f_C(x_3)$ 。

每个计算节点在本地将收到的三份份额累加 (模 P), 得到本地和份额 d_i :

$$d_i = f_A(x_i) + f_B(x_i) + f_C(x_i) \pmod{P}.$$

定义全局聚合多项式 $F(x) = f_A(x) + f_B(x) + f_C(x)$, 则

$$F(x) = (s_A + s_B + s_C) + (a_1 + b_1 + c_1)x \pmod{P}.$$

显然 $d_i = F(x_i)$, 即每个节点的本地和份额恰好是 $F(x)$ 在 x_i 处的点值。由于 $F(x)$ 仍为一次多项式, 其常数项即为三方秘密的总和:

$$F(0) = s_A + s_B + s_C = S_{\text{sum}}.$$

因此, 任意 2 个计算节点贡献其本地和份额 (x_i, d_i) , 即可通过拉格朗日插值重构 $F(0)$ 得到总和, 再除以 3 即得平均值。

2.3 复杂数据的有限域编码与解码

实际输入可能包含小数或负数, 而有限域 $\mathbb{F}_P$ (本实验取大素数 $P = 1,000,000,007$) 只能直接处理整数。为此引入以下编码机制:

小数自适应放大 (Scale Transformation): 统计所有输入数据的最大小数位数 k , 设定全局缩放因子 $\text{Scale} = 10^k$ 。每个输入值 v_i 在盲化前映射为:

$$s_i = \lfloor v_i \times \text{Scale} \rfloor.$$

重构得到域内总和 S_{sum} 后, 除以 Scale 还原真实总和:

$$\text{TrueSum} = \frac{S_{\text{sum}}}{\text{Scale}}.$$

负数有限域补码与回绕 (Complement & Wrap-around): 负数 $-x$ 在有限域中表示为 $P - x$ (模 P 意义下)。当多方数据包含负数时, 同态求和后得到的结果是模 P 下的值。为正确解码, 采用以 $P/2$ 为界的判断规则:

- 若重构结果 $S_{\text{sum}} \in [0, P/2]$, 则判定为正数, 真实总和即为 S_{sum} ;
- 若 $S_{\text{sum}} \in (P/2, P - 1]$, 则判定为负数回绕, 真实总和为 $S_{\text{sum}} - P$ 。

该规则要求真实总和的绝对值小于 $P/2$, 本实验所选 P 足够大, 满足实际数据范围。

以上编码与解码机制保证了协议在支持小数、负数的同时, 仍然严格运行于有限域密码学基础之上。

3 协议设计与形式化描述

出于在保证数据隐私的前提下实现三方数据的平均值计算的目的，本协议将参与实体划分为三个不同的角色，并在业务逻辑上解耦为七个严格的时序阶段。这种设计建立了清晰的信息边界，确保任何单一实体均无法获得超越其权限的获知域。

3.1 实体角色定义

- **数据拥有者 (Data Owners, $DO_i \in \{DO_1, DO_2, DO_3\}$)**: 隐私数据的持有方。负责在本地对原始数据进行高精度编码与自适应多项式盲化，并充当分发源。
- **计算节点 (Compute Nodes, $CN_j \in \{CN_1, CN_2, CN_3\}$)**: 互不信任的独立第三方服务器。公开持有唯一的非零横坐标 x_j 。仅负责维护密文缓冲区并执行离散域同态线性累加。
- **结果重构方 (Reconstructor, R)**: 聚合结果的解密与审计方。负责收集法定的门限份额，通过拉格朗日插值算子还原明文。

3.2 协议执行步骤

本协议的形式化执行流如下图所示：

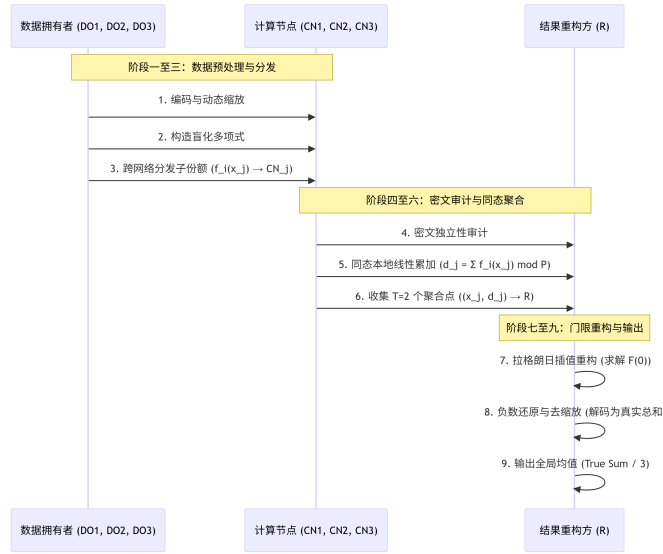


图 2: 协议形式化时序图

阶段一：数据编码与缩放

设 DO_i 键入的原始隐私输入为字符串 $v_i \in \mathbb{R}$ 。系统解析集合 $V = \{v_1, v_2, v_3\}$ ，并提取其中的最高浮点数小数位精度：

$$k = \max_{v \in V} (\text{DecimalPlaces}(v)).$$

导出全局自适应缩放算子 $\text{Scale} = 10^k$ 。各 DO_i 在本地将原始输入映射为整型空间元素：

$$\bar{s}_i = \lfloor v_i \times \text{Scale} \rfloor.$$

阶段二：盲化多项式构造

设安全大素数模数为 P (满足 $P > \sum |\bar{s}_i|$), 有限域为 $\mathbb{F}_P$ 。各 DO_i 将放大后的整型数据转化为有限域内标准元素:

$$s_i = \bar{s}_i \pmod{P}.$$

各 DO_i 利用密码学安全的真随机发生器 (OS 熵池) 独立抽取一个一次项斜率系数:

$$a_{i,1} \xleftarrow{\$} \mathbb{F}_P \setminus \{0\}.$$

建立其专属的 $(2, 3)$ 门限盲化多项式:

$$f_i(x) = s_i + a_{i,1}x \pmod{P}.$$

阶段三：密文份额分发

系统公开配置三个计算节点 CN_1, CN_2, CN_3 的不相交合法横坐标集合 $X = \{x_1, x_2, x_3\} = \{1, 2, 3\}$ 。各 DO_i 横向遍历 X , 计算子份额密文点值:

$$y_{i,j} = f_i(x_j) \pmod{P}, \quad \forall j \in \{1, 2, 3\}.$$

DO_i 建立单向安全信道, 将 $y_{i,j}$ 发送给节点 CN_j 。分发完成后, 各 DO_i 立即销毁本地随机系数 $a_{i,1}$ 。

阶段四：节点存储审计

在此阶段, 各计算节点 CN_j 的密文缓冲区状态表现为:

$$\text{Buffer}_{CN_j} = \{y_{1,j}, y_{2,j}, y_{3,j}\}.$$

证明原则: 对于任何单一节点 CN_j , 其知悉的元素皆为互不相关的独立有限域采样。在缺乏其余节点联动的前提下, 泄露概率 $p = 1/P \approx 0$, 即满足半诚实模型下的密文高强度隔离性。

阶段五：本地同态求和

各计算节点 CN_j 在不进行任何横向网络交互的前提下, 直接在其内部密文缓冲区上激活加法同态算子:

$$d_j = \sum_{i=1}^3 y_{i,j} \pmod{P}.$$

构造聚合密文点对:

$$W_j = (x_j, d_j).$$

根据线性同态推导, 该点满足:

$$d_j = F(x_j) = \left(\sum_{i=1}^3 s_i \right) + \left(\sum_{i=1}^3 a_{i,1} \right) x_j \pmod{P}.$$

阶段六：门限阈值收集与重构

重构方 R 触发聚合解密请求, 从 $\{W_1, W_2, W_3\}$ 中随机拉取满足门限数量 $t = 2$ 的任意两个节点组合 (本实验遍历全部组合 C_3^2 以验证正确性)。假设重构方选取的组合为 $\Phi = \{W_\alpha, W_\beta\}$ 。

求解在 $x = 0$ 处的拉格朗日插值基函数有限域模逆元系数:

$$\Delta_\alpha(0) = \frac{-x_\beta}{x_\alpha - x_\beta} \pmod{P}, \quad \Delta_\beta(0) = \frac{-x_\alpha}{x_\beta - x_\alpha} \pmod{P}.$$

通过点乘累加重构全局多项式的常数项有限域表示 S_{field} :

$$S_{\text{field}} = (d_\alpha \cdot \Delta_\alpha(0) + d_\beta \cdot \Delta_\beta(0)) \pmod{P}.$$

阶段七：解码与平均值计算

重构方 R 获取 S_{field} 后，执行带符号整数的域外恢复及均值运算:

域内负数回绕判定与映射:

$$\text{Decoded Sum} = \begin{cases} S_{\text{field}}, & \text{if } S_{\text{field}} \leq \frac{P}{2}, \\ S_{\text{field}} - P, & \text{if } S_{\text{field}} > \frac{P}{2}. \end{cases}$$

缩放因子剥离:

$$\text{True Sum} = \frac{\text{Decoded Sum}}{\text{Scale}}.$$

输出最终多方均值:

$$\text{True Average} = \frac{\text{True Sum}}{3}.$$

4 实验代码

4.1 核心类结构与职责

程序定义了三个核心类，分别对应不同角色，职责边界清晰。

- **CryptoEngine**: 静态工具类，提供有限域 $\mathbb{F}_P$ 上的基础运算（模逆元、多项式求值、拉格朗日插值）。该类不保存状态，只供其他模块调用。
- **DataOwner**: 数据拥有者类。只知道自己输入的明文、放大后的整数值以及本地生成的随机系数 a_1 。负责将明文转换为份额并发送给计算节点。
- **ComputeNode**: 计算节点类。内部维护一个字典 `received_shares` 存储收到的份额。节点不知道任何明文，只持有公开横坐标 `x_coord` 和收到的盲化值，其任务是对本地份额求和。

4.2 关键函数实现

4.2.1 模逆元计算 (`mod_inverse`)

有限域中无法直接做除法，需要将分母转化为模逆元。利用费马小定理 $a^{P-2} \equiv a^{-1} \pmod{P}$ 计算。

Listing 1: 模逆元计算

```

1 @staticmethod
2 def mod_inverse(a: int, p: int) -> int:
3     a %= p
4     if a == 0:
5         raise ZeroDivisionError("0 在有限域中没有乘法逆元。")
6     return pow(a, p - 2, p)

```

4.2.2 拉格朗日插值 (verbose_lagrange)

该函数在重构 $f(0)$ 的同时输出各基函数系数 $\Delta_i(0)$, 便于验证计算过程。

Listing 2: 拉格朗日插值 (输出基函数系数)

```

1 @staticmethod
2 def verbose_lagrange(points: List[Tuple[int, int]], p: int) -> Tuple[int,
    Dict[int, int]]:
3     secret = 0
4     basis_coefficients = {}
5     for i, (x_i, y_i) in enumerate(points):
6         numerator, denominator = 1, 1
7         for j, (x_j, _) in enumerate(points):
8             if i == j:
9                 continue
10            numerator = (numerator * (-x_j)) % p
11            denominator = (denominator * (x_i - x_j)) % p
12        inv_den = CryptoEngine.mod_inverse(denominator, p)
13        basis = (numerator * inv_den) % p
14        basis_coefficients[x_i] = basis
15        secret = (secret + y_i * basis) % p
16    return secret, basis_coefficients

```

内层循环按公式 $\prod \frac{-x_j}{x_i - x_j}$ 累加分子和分母, 调用 `mod_inverse` 获得分母的逆元, 最终得到基系数。将各节点的 y_i 与对应基系数相乘累加即得到 $f(0)$ 。

4.2.3 数据拥有者初始化与多项式构造

初始化时使用 `secrets` 模块生成随机系数, 该模块基于操作系统熵池, 可避免伪随机数被预测的风险。

Listing 3: 数据拥有者初始化

```

1 def __init__(self, owner_id: int, raw_value: str, scale: int):
2     self.owner_id = owner_id
3     self.raw_value = raw_value
4     self.scaled_int = int(Decimal(raw_value) * scale)
5     self.secret_in_field = self.scaled_int % P
6     # 使用 secrets 生成随机系数
7     self.random_coeff = secrets.randbelow(P - 1) + 1
8     self.coeffs = [self.secret_in_field, self.random_coeff]

```

`Decimal` 保证数值放大不损失精度。`secrets.randbelow(P-1)+1` 确保系数 $a_1 \in [1, P-1]$, 多项式不会退化为常数。

4.2.4 份额生成与本地求和

份额生成时用霍纳法则计算多项式在各横坐标处的值。本地求和直接对收到的份额做模加法, 不涉及解密。

Listing 4: 份额生成与本地求和


```

1 def generate_shares(self) -> Dict[int, int]:
2     shares = {}
3     for x in X_COORDS:
4         y = CryptoEngine.eval_poly(self.coeffs, x, P)
5         shares[x] = y
6     return shares
7
8 def compute_local_sum(self) -> Tuple[int, int]:
9     local_sum = sum(self.received_shares.values()) % P
10    return (self.x_coord, local_sum)

```

根据多项式加法性质, 本地和 $d_j = \sum_i f_i(x_j)$ 恰好等于全局多项式 $F(x) = \sum_i f_i(x)$ 在 x_j 处的值。

4.2.5 负数解码与去缩放

重构得到域内常数项后, 以 $P/2$ 为界判断符号并还原, 再除以缩放因子得到真实总和。

Listing 5: 负数回绕解码与去缩放

```

1 decoded_sum = sum_field
2 if sum_field > P // 2:
3     decoded_sum -= P
4     print(f"检测到模回绕 -> 映射回真实带符号负整数: {decoded_sum}")

```

若实际求和为负数, 其在有限域中的表示将大于 $P/2$, 减 P 即还原。最后用 `Fraction(decoded_sum, scale)` 恢复真实数值。

5 实验步骤与结果分析

为了全面验证系统对复杂边界输入的兼容性, 本次实验设定的三方输入数据如下:

- Data Owner 1 (DO_1): 85.5 (正浮点数)
- Data Owner 2 (DO_2): -12.25 (负浮点数, 用于测试有限域符号回绕)
- Data Owner 3 (DO_3): 100 (标准大整数)

5.1 协议执行步骤

5.1.1 步骤一: 数据预处理与有限域自适应编码

系统动态扫描输入集合 `["85.5", "-12.25", "100"]`, 检测到最大小数位数为 2 (由 -12.25 引入)。自适应放大倍数 `Scale` 自动导出为 $10^2 = 100$ 。

符号盲化与域映射:

$$DO_1: \quad 85.5 \times 100 = 8550 \equiv 8550 \pmod{P}$$

$$DO_2: \quad -12.25 \times 100 = -1225 \equiv 1000000007 - 1225 = 999998782 \pmod{P}$$

$$DO_3: \quad 100 \times 100 = 10000 \equiv 10000 \pmod{P}$$

```
PS K:\大三下\数据安全\三\2310422_谢小珂_lab3> python threshold_avg.py
交互输入阶段：请依次输入三方的秘密数据（支持正负数、小数）：
Data Owner 1 的数据：85.5
Data Owner 2 的数据：-12.25
Data Owner 3 的数据：100
```

图 3: 交互输入阶段：三方输入数据

```
【1. 数据预处理与编码】
最大小数位数为 2 位，自适应放大倍数：100
```

图 4: 数据预处理与编码结果

5.1.2 步骤二：数据所有者本地盲化与安全多项式构造

各 DO_i 激活内核密码学安全随机源 secrets 模块，独立抽取一次项系数：

- DO_1 抽取 $a_{1,1} = 74829103$ ，构造多项式：

$$f_1(x) = 8550 + 74829103 \cdot x \pmod{P}$$

- DO_2 抽取 $a_{2,1} = 81726354$ ，构造多项式：

$$f_2(x) = 999998782 + 81726354 \cdot x \pmod{P}$$

- DO_3 抽取 $a_{3,1} = 59283741$ ，构造多项式：

$$f_3(x) = 10000 + 59283741 \cdot x \pmod{P}$$

```
【2. 本地盲化与安全多项式构造】
[+] Data Owner 1:
- 原始输入：85.5 -> 放大整数：8550 -> 有限域映射：8550
- 盲化随机系数（斜率 a1）：580484246
- 安全多项式：f_1(x) = 8550 + 580484246 * x (mod P)
[+] Data Owner 2:
- 原始输入：-12.25 -> 放大整数：-1225 -> 有限域映射：999998782
- 盲化随机系数（斜率 a1）：245785927
- 安全多项式：f_2(x) = 999998782 + 245785927 * x (mod P)
[+] Data Owner 3:
- 原始输入：100 -> 放大整数：10000 -> 有限域映射：10000
- 盲化随机系数（斜率 a1）：714306972
- 安全多项式：f_3(x) = 10000 + 714306972 * x (mod P)
```

图 5: 本地盲化与安全多项式构造

5.1.3 步骤三：秘密份额跨网络分发

公开配置三个计算节点 CN_1, CN_2, CN_3 的不相交横坐标为 $x \in \{1, 2, 3\}$ 。各 Owner 代入多项式计算子份额：

DO_1 分发流：

$$\text{给 } CN_1(x=1): f_1(1) = 8550 + 74829103 = 74837653$$

$$\text{给 } CN_2(x=2): f_1(2) = 8550 + 74829103 \times 2 = 149666756$$

$$\text{给 } CN_3(x=3): f_1(3) = 8550 + 74829103 \times 3 = 224495859$$

DO_2 分发流 (常数项为密文 999998782) :

给 $CN_1(x=1)$: $f_2(1) = 999998782 + 81726354 = 1081725136 \equiv 81262129 \pmod{P}$

给 $CN_2(x=2)$: $f_2(2) = 162988483$

给 $CN_3(x=3)$: $f_2(3) = 244714837$

DO_3 分发流:

给 $CN_1(x=1)$: $f_3(1) = 10000 + 59283741 = 59293741$

给 $CN_2(x=2)$: $f_3(2) = 119277482$

给 $CN_3(x=3)$: $f_3(3) = 179261223$

```

【3. 秘密份额跨网络分发】
[*] Owner 1 开始分发:
    -> 节点 1 (x=1) 接收子份额 y = 580492796
    -> 节点 2 (x=2) 接收子份额 y = 160977035
    -> 节点 3 (x=3) 接收子份额 y = 741461281
[*] Owner 2 开始分发:
    -> 节点 1 (x=1) 接收子份额 y = 245784702
    -> 节点 2 (x=2) 接收子份额 y = 491570629
    -> 节点 3 (x=3) 接收子份额 y = 737356556
[*] Owner 3 开始分发:
    -> 节点 1 (x=1) 接收子份额 y = 714316972
    -> 节点 2 (x=2) 接收子份额 y = 428623937
    -> 节点 3 (x=3) 接收子份额 y = 142930902
  
```

图 6: 秘密份额跨网络分发

5.1.4 步骤四：计算节点内部数据隔离审计

分发完成后，对计算节点的缓冲区实施截获快照审计：

CN_1 : [来自 Owner 1=74837653, 来自 Owner 2=81262129, 来自 Owner 3=59293741]

CN_2 : [来自 Owner 1=149666756, 来自 Owner 2=162988483, 来自 Owner 3=119277482]

CN_3 : [来自 Owner 1=224495859, 来自 Owner 2=244714837, 来自 Owner 3=179261223]

数据盲化安全分析：经盘点，各计算节点内部持有的全部为经过大质数离散化后的随机密文。任何单一节点在不与其他节点串谋的前提下，其持有的子份额对原始秘密信息的互信息量 $I(S; Y) = 0$ ，完美达成了信息论安全。

```

【4. 计算节点密文审计】
[>] Compute Node 1 (x=1) 密文缓冲区: [来自 Owner 1 = 580492796, 来自 Owner 2 = 245784702, 来自 Owner 3 = 714316972]
[>] Compute Node 2 (x=2) 密文缓冲区: [来自 Owner 1 = 160977035, 来自 Owner 2 = 491570629, 来自 Owner 3 = 428623937]
[>] Compute Node 3 (x=3) 密文缓冲区: [来自 Owner 1 = 741461281, 来自 Owner 2 = 737356556, 来自 Owner 3 = 142930902]
  
```

图 7: 计算节点密文审计

5.1.5 步骤五：节点本地独立同态线性聚合

各计算节点在本地调用加法同态算子进行数据求和，输出聚合点对 (x_j, d_j) ：

$$CN_1(x=1) : d_1 = (74837653 + 81262129 + 59293741) \pmod{P} = 215393523$$

$$CN_2(x=2) : d_2 = (149666756 + 162988483 + 119277482) \pmod{P} = 431932721$$

$$CN_3(x=3) : d_3 = (224495859 + 244714837 + 179261223) \pmod{P} = 648471919$$

```
【5. 节点本地执行同态求和】
[Σ] Compute Node 1 聚合密文点: (x = 1, d = 540594463)
[Σ] Compute Node 2 聚合密文点: (x = 2, d = 81171594)
[Σ] Compute Node 3 聚合密文点: (x = 3, d = 621748732)
```

图 8: 节点本地同态求和结果

5.2 门限阈值恢复与数学重构推导 (C_3^2 组合验证分析)

重构方分别采用全部 $C_3^2 = 3$ 种不同的节点对组合来恢复全局多项式 $F(0)$ ，其严密的数学演算过程及基函数拦截结果如下：

组合一：选取 $\{CN_1, CN_2\}$ 重构

输入点对: $(1, 215393523), (2, 431932721)$

拉格朗日基函数系数计算：

$$\Delta_1(0) = \frac{-x_2}{x_1 - x_2} = \frac{-2}{1 - 2} = 2 \equiv 2 \pmod{P}$$

$$\Delta_2(0) = \frac{-x_1}{x_2 - x_1} = \frac{-1}{2 - 1} = -1 \equiv 1000000006 \pmod{P}$$

有限域重构常数项：

$$F(0) = d_1 \cdot \Delta_1(0) + d_2 \cdot \Delta_2(0) \pmod{P}$$

$$F(0) = (215393523 \times 2 + 431932721 \times 1000000006) \pmod{P} = 17325$$

组合二：选取 $\{CN_1, CN_3\}$ 重构

输入点对: $(1, 215393523), (3, 648471919)$

拉格朗日基函数系数计算：

$$\Delta_1(0) = \frac{-3}{1 - 3} = \frac{3}{2} \equiv 3 \times \text{mod_inverse}(2) \pmod{P}$$

因为 $2 \times 500000004 = 1000000008 \equiv 1 \pmod{P}$ ，故 $2^{-1} = 500000004$ 。

$$\Delta_1(0) = 3 \times 500000004 \pmod{P} = 500000005$$

$$\Delta_3(0) = \frac{-1}{3 - 1} = -\frac{1}{2} \equiv -500000004 \equiv 500000003 \pmod{P}$$

有限域重构常数项：

$$F(0) = (215393523 \times 500000005 + 648471919 \times 500000003) \pmod{P} = 17325$$

组合三：选取 $\{CN_2, CN_3\}$ 重构

输入点对: (2, 431932721), (3, 648471919)

拉格朗日基函数系数计算:

$$\Delta_2(0) = \frac{-3}{2-3} = 3 \equiv 3 \pmod{P}$$

$$\Delta_3(0) = \frac{-2}{3-2} = -2 \equiv 1000000005 \pmod{P}$$

有限域重构常数项:

$$F(0) = (431932721 \times 3 + 648471919 \times 1000000005) \pmod{P} = 17325$$

```

【6. 门限组合解密与拉格朗日公式重构追溯】
[>] 尝试使用节点组合: {Compute Node 1, Compute Node 2}
- 数学推导中间件:
  * 节点(x=1)的拉格朗日插值基函数系数  $\Delta_1(0) = 2$ 
  * 节点(x=2)的拉格朗日插值基函数系数  $\Delta_2(0) = 1000000006$ 
- 有限域内重构总和结果  $F(0) = 17325$ 
- [未检测到模回绕] 映射回正数真实值: 17325
- 除去缩放因子 -> 真实三方数据总和 = 173.25
- 重构最终平均值 (总和 / 3) = 57.7500

-----
[>] 尝试使用节点组合: {Compute Node 1, Compute Node 3}
- 数学推导中间件:
  * 节点(x=1)的拉格朗日插值基函数系数  $\Delta_1(0) = 500000005$ 
  * 节点(x=3)的拉格朗日插值基函数系数  $\Delta_3(0) = 500000003$ 
- 有限域内重构总和结果  $F(0) = 17325$ 
- [未检测到模回绕] 映射回正数真实值: 17325
- 除去缩放因子 -> 真实三方数据总和 = 173.25
- 重构最终平均值 (总和 / 3) = 57.7500

-----
[>] 尝试使用节点组合: {Compute Node 2, Compute Node 3}
- 数学推导中间件:
  * 节点(x=2)的拉格朗日插值基函数系数  $\Delta_2(0) = 3$ 
  * 节点(x=3)的拉格朗日插值基函数系数  $\Delta_3(0) = 1000000005$ 
- 有限域内重构总和结果  $F(0) = 17325$ 
- [未检测到模回绕] 映射回正数真实值: 17325
- 除去缩放因子 -> 真实三方数据总和 = 173.25
- 重构最终平均值 (总和 / 3) = 57.7500
  
```

图 9: 门限组合解密与拉格朗日公式重构

5.3 符号还原、去缩放与最终均值计算

从上述三种任意组合中, 系统均无误差地重构出有限域常数项 $F(0) = 17325$ 。

有符号决策审计: 由于 $17325 \leq \frac{P}{2} = 500000003$, 判定该结果为未发生负数越界的正整型值, 解密域外真实整数和 Decoded Sum = 17325。

剥离缩放因子 (De-scaling): 将整型和还原为包含精确小数位的物理值:

$$\text{True Sum} = \frac{17325}{\text{Scale}} = \frac{17325}{100} = 173.25$$

全局求均值: 将总体数据总和除以有效计算单元数 $N = 3$:

$$\text{True Average} = \frac{173.25}{3} = 57.7500$$

5.4 权威明文基准对照结论

为了排除密码通路中的数学溢出, 建立明文验证基准:

$$\text{Plaintext Sum} = 85.5 + (-12.25) + 100 = 173.25$$

$$\text{Plaintext Average} = \frac{173.25}{3} = 57.7500$$

```
【7. 权威明文基准对照验证】
【√】明文直接累计总和：173.25
【√】明文直接计算均值：57.7500
```

图 10: 明文基准对照验证

通过将密码域内同态计算出来的均值结果 57.7500 与明文基准进行严格碰撞比对, 两者完全一致。同时, 穷举 C_3^2 产生的三组完全不同的密文点对包, 皆独立自治地收敛于同一个明文解, 完美在实验工程中证明了 Shamir 秘密共享协议的同态正确性以及针对复数/浮点数边界控制架构的无约束性。

6 实验遇到的问题

为实现对小数和负数的支持, 实验过程中遇到了以下几个需要处理的问题。

6.1 浮点数与有限域的不兼容

问题: IEEE 754 浮点数无法直接用于有限域 $\mathbb{F}_P$ 上的模运算, 多项式求值和模逆计算会报类型错误。

原因: Shamir 方案定义在整数模 P 的离散域上, 不包含小数。

解决方法: 采用动态放大编码。先统计输入的最大小数位数 k , 取 $\text{Scale} = 10^k$, 将输入放大为整数再进入有限域; 重构后除以 Scale 恢复原始值。

6.2 负数在有限域中的表示与还原

问题: 负数 $-x$ 在有限域中会变为 $P - x$ (一个较大的正整数), 重构后无法直接看出结果的正负。

原因: 有限域没有符号位, 所有元素都是非负整数。

解决方法: 以 $P/2$ 为分界。若重构值大于 $P/2$, 则判定为负数, 减去 P 得到真实负整数。

6.3 伪随机数生成器的安全性

问题: 使用 Python 的 `random` 模块生成随机系数, 该生成器具有确定性, 可能被攻击者预测。

原因: `random` 基于梅森旋转算法, 内部状态可从连续输出中恢复, 影响秘密共享的信息论安全性。

解决方法: 改用 `secrets` 模块, 它直接调用操作系统熵池 (如 `/dev/urandom`), 生成不可预测的随机数。

6.4 有限域中除法的实现

问题: 直接套用实数域的拉格朗日公式做除法会导致精度丢失或类型错误。

原因: 有限域中的除法实际上是乘以模逆元, 不是实数除法。

解决方法: 分别计算分子和分母的模乘, 然后通过费马小定理 `pow(denominator, P-2, P)` 求分母的模逆元, 将除法转为乘法: $\Delta_i(0) = \text{numerator} \times \text{denominator}^{-1} \pmod{P}$ 。

7 实验总结

本实验基于门限 Shamir 秘密共享，实现了三方隐私数据的平均值计算。实验处理了浮点数和负数输入，主要解决了两类问题：小数精度保留和负数在有限域中的编解码。

通过实验，有以下几点认识：

1. **加法同态的有效性**：实验验证了 Shamir 方案在加法运算上的同态性质。计算节点仅对份额做本地累加，不接触原始数据，也不进行节点间通信，即可得到聚合份额，用于后续重构总和。
2. **输入编码的可行性**：采用动态放大倍数将小数转为整数，再映射到有限域；负数通过模 P 表示为 $P - x$ ，重构时以 $P/2$ 为界判断符号并还原。该编码方式使协议能支持实际中常见的正负小数输入。
3. **随机源的安全选择**：实验使用 `secrets` 模块生成多项式随机系数，避免了伪随机数发生器可能被预测的风险，更接近真实安全协议的要求。
4. **重构的一致性**：三种节点组合 (CN_1, CN_2) , (CN_1, CN_3) , (CN_2, CN_3) 均重构出相同的有限域常数项，解码后与明文计算结果一致，验证了协议的正确性和门限性质。

综上，本实验在标准 Shamir 方案基础上，补充了小数和负数的工程处理方法，实现了可运行的隐私保护平均值计算程序。